Частное образовательное учреждение высшего образования
«Санкт-Петербургский институт экономики и управления»
(СПбИЭУ)

Специальность 09.02.07 «Информационные системы и программирование»
ПМ.03 Сопровождение и обслуживание программного обеспечения компьютерных систем
МДК.03.01 Внедрение и поддержка компьютерных систем

ОТЧЁТ

по промежуточному контролю № 19

Тема 1.1. Основные методы внедрения и анализа функционирования ПО
«Разработка утилит развёртывания, аудита и генерации отчётности на C#»
Вариант 10

Выполнил: студент группы И14-23-9
Иванов Егор Михайлович

Проверила: преподаватель
Чудинова Александра Анатольевна
Оценка: _____

Санкт-Петербург
2026

1. Описание задания

Цель работы: закрепить навыки автоматизации, проверки окружения, работы с внешними сервисами и генерации эксплуатационной документации с использованием C# .NET 8 и интеграции с реальными публичными API.

Вариант 10. Генератор суточных отчётов по транзакциям питания в столовой. Создать C#-программу, которая симулирует выгрузку данных о транзакциях с публичного мок-сервера, суммирует финансовые показатели по категориям и создаёт отчёт в формате CSV. Публичный источник: `https://jsonplaceholder.typicode.com/posts`.

Общие методические указания: проект Console App (.NET 8) в MS Visual Studio; запросы через стандартный класс `HttpClient`; обработка сетевых ошибок (try-catch, таймауты); результат обязательно сохраняется в текстовый файл (.md или .csv) — подход Docs-as-Code.

Требования из теоретического материала (занятие № 19), учтённые в программе

Таблица 1 – Связь с теорией

Требование лекции	**Как реализовано**
Отчётная форма «Ведомость учёта питания»: поля Date, Category (бюджет / коммерция), Amount, Transaction_Status	колонки CSV-реестра Date, Category, Amount, Transaction_Status; категории «Бюджет» и «Коммерция»
Полнота данных: точные метки времени, идентификаторы АРМ и оператора	в каждой строке — дата, время, StationId, OperatorId, номер транзакции
Защита от изменений	контрольные суммы SHA-256 всех файлов отчёта; журнал открывается только на дозапись
Мультиформатный экспорт (печать/архив + аналитика)	CSV для Excel (UTF-8 с BOM, «;», запятая в числах) + сводка Markdown
Конфигурация: appsettings.json → переменные окружения → аргументы	AppConfig: appsettings.json → CAMPUS_STATION_ID, CAMPUS_OPERATOR_ID → --date, --source

2. Описание решения

Таблица 2 – Структура проекта Campus.MealDailyReport

Файл	**Назначение**
Program.cs	порядок работы: конфигурация → выгрузка → симуляция → итоги → файлы отчёта; коды завершения
AppConfig.cs	сбор настроек по иерархии appsettings.json → CAMPUS_* → аргументы
MockServerClient.cs	HttpClient: таймаут, 3 попытки с паузой 1 с / 2 с, кэш последней

	выгрузки
TransactionSimulator.cs	детерминированная симуляция транзакций кассы и суммирование по категориям
MealTransaction.cs	модели: транзакция, категория, статус, итоги категории
ReportWriter.cs	CSV-реестр, CSV-итоги, Markdown-сводка, файл SHA-256
ReportLog.cs	журнал: консоль оператора + файл meal_report.log (append-only)

Правила симуляции. Пользователи мок-сервера 1–4 — студенты-бюджетники (дотация до 150 ₽ за покупку), остальные — коммерческое питание. Время покупки распределяется по часам работы столовой (08:00–19:00), комплекс зависит от времени: завтрак 95 ₽, обед 185 ₽ или полдник 70 ₽, плюс 0–4 дополнительные позиции по 15 ₽. Каждая 17-я запись — отказ, каждая 23-я — возврат. Преобразование детерминированное: одна выгрузка за одни сутки всегда даёт одинаковый отчёт, поэтому совпадают и контрольные суммы при сверке.

3. Программный код с комментариями

Исходный код решения и документация в репозитории GitHub: https://github.com/Egorvan606-senior/mdk03/tree/main/pk-19

Листинг 1 – Program.cs

```
// ============================================================================
// campus-meal-daily-report — генератор суточных отчётов по транзакциям питания
// ИС «Умный кампус», модуль «Столовая». Промежуточный контроль № 19, вариант 10.
// Автор: Иванов Е.М., группа И14-23-9.
//
// Порядок работы:
//   1. Сбор конфигурации: appsettings.json -> переменные CAMPUS_* -> аргументы.
//   2. Выгрузка «транзакций» с публичного мок-сервера jsonplaceholder.typicode.com/posts
//      через HttpClient (таймаут, повторы, кэш при недоступности сети).
//   3. Симуляция транзакций кассы за отчётные сутки.
//   4. Суммирование финансовых показателей по категориям (бюджет / коммерция).
//   5. Сохранение отчёта: CSV (реестр и итоги) + Markdown-сводка + SHA-256.
//
// Запуск:  dotnet run -- --date 2026-10-24
// Коды завершения: 0 — успех; 1 — ошибка; 2 — отчёт сформирован по кэшу
(предварительный).
// ============================================================================

using System.Globalization;
using System.Text;
using Campus.MealDailyReport;

Console.OutputEncoding = Encoding.UTF8;
var ru = CultureInfo.GetCultureInfo("ru-RU");

AppConfig cfg;
try
{
    cfg = AppConfig.Build(AppContext.BaseDirectory, args);
}
catch (ArgumentException ex)
{
    Console.Error.WriteLine($"Ошибка параметров: {ex.Message}");
    return 1;
```

```
}

// Каждые сутки — отдельная папка: reports\2026-10-24\...
string dayDir = Path.GetFullPath(Path.Combine(cfg.OutputDir,
cfg.ReportDate.ToString("yyyy-MM-dd")));
using var log = new ReportLog(Path.GetFullPath(Path.Combine(cfg.OutputDir, "logs")));
using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); }; // Ctrl+C —
корректная остановка

log.Raw("======================================================================",
ConsoleColor.Cyan);
log.Raw("  ИС «Умный кампус» · Суточный отчёт по транзакциям питания · v1.0.0",
ConsoleColor.Cyan);
log.Raw($"  Сутки: {cfg.ReportDate.ToString("dd MMMM yyyy (dddd)", ru)}",
ConsoleColor.Cyan);
log.Raw($"  АРМ: {cfg.StationId} · оператор: {cfg.OperatorId} · ПК:
{Environment.MachineName}", ConsoleColor.Cyan);
log.Raw("======================================================================",
ConsoleColor.Cyan);

try
{
    // ---- 1. Выгрузка с мок-сервера ----------------------------------------
    using var http = new HttpClient { Timeout =
TimeSpan.FromSeconds(cfg.TimeoutSeconds) };
    http.DefaultRequestHeaders.UserAgent.ParseAdd("SmartCampus-MealDailyReport/1.0");
    string cachePath = Path.GetFullPath(Path.Combine(cfg.OutputDir, "cache",
"posts_cache.json"));
    var loaded = await new MockServerClient(http, log, cachePath,
cfg.Retries).LoadAsync(cfg.SourceUrl, cts.Token);

    if (loaded.Posts.Count == 0)
    {
        log.Error("Выгрузка пуста — отчёт не формируется");
        return 1;
    }

    // ---- 2. Симуляция транзакций ------------------------------------------
    var transactions = loaded.Posts
        .Select(p => TransactionSimulator.Create(p, cfg.ReportDate, cfg.StationId,
cfg.OperatorId))
        .OrderBy(t => t.Timestamp)
        .ThenBy(t => t.TransactionId)
        .ToList();

    // Контроль полноты: номера транзакций уникальны (повтор = двойная выгрузка, суммы не
сойдутся)
    var duplicates = transactions.GroupBy(t => t.TransactionId).Where(g => g.Count() >
1).Select(g => g.Key).ToList();
    if (duplicates.Count > 0)
    {
        log.Error($"Дубликаты транзакций: {string.Join(", ", duplicates)} — отчёт не
формируется");
        return 1;
    }

    log.Ok($"Сформировано транзакций за сутки: {transactions.Count}");

    // ---- 3. Суммирование по категориям ------------------------------------
    var totals = TransactionSimulator.Summarize(transactions);

    // ---- 4. Сохранение отчёта (Docs-as-Code) ------------------------------
    string stamp = cfg.ReportDate.ToString("yyyyMMdd");
    string header = $"Суточный отчёт по питанию за {cfg.ReportDate:dd.MM.yyyy}; АРМ
{cfg.StationId}; оператор {cfg.OperatorId}; " +
                    $"сформирован {DateTime.Now:dd.MM.yyyy HH:mm}; " +
                    (loaded.FromCache ? $"ПРЕДВАРИТЕЛЬНЫЙ (кэш от
{loaded.LoadedAt:dd.MM.yyyy HH:mm})" : $"источник {cfg.SourceUrl}");

    var writer = new ReportWriter(dayDir);
    writer.WriteTransactions($"meal_transactions_{stamp}.csv", transactions);
    writer.WriteCategoryReport($"meal_daily_report_{stamp}.csv", totals, header);
    writer.WriteMarkdownSummary($"meal_daily_report_{stamp}.md", cfg, loaded, totals,
transactions.Count);
```

```
    string sums = writer.WriteChecksums($"meal_daily_report_{stamp}.sha256");

    // ---- 5. Итоги на экран оператора ----------------------------------------
    log.Raw("");
    log.Raw($"{"Категория",-11}{"Успешно",8}{"Возвр.",7}{"Отказ",7}{"Выручка",12}
{"Дотация",11}{"Оплачено",12}{"Ср.чек",9}", ConsoleColor.White);
    log.Raw(new string('—', 77), ConsoleColor.DarkGray);
    foreach (var c in totals)
    {
        log.Raw($"{c.Category.ToRussian(),-11}{c.Success,8}{c.Refunds,7}{c.Declined,7}
{c.NetRevenue.ToString("N2", ru),12}" +
                $"{c.Subsidy.ToString("N2", ru),11}{c.Paid.ToString("N2", ru),12}
{c.AverageCheck.ToString("N2", ru),9}");
    }

    log.Raw(new string('—', 77), ConsoleColor.DarkGray);
    log.Raw($"{"ИТОГО",-11}{totals.Sum(c => c.Success),8}{totals.Sum(c => c.Refunds),7}
{totals.Sum(c => c.Declined),7}" +
            $"{totals.Sum(c => c.NetRevenue).ToString("N2", ru),12}{totals.Sum(c =>
c.Subsidy).ToString("N2", ru),11}" +
            $"{totals.Sum(c => c.Paid).ToString("N2", ru),12}", ConsoleColor.White);
    log.Raw("");
    log.Ok($"Отчёт сохранён: {dayDir}");
    log.Info($"Контрольные суммы: {Path.GetFileName(sums)} · журнал: {log.FilePath}");

    if (loaded.FromCache)
    {
        log.Warn("Отчёт ПРЕДВАРИТЕЛЬНЫЙ (по кэшу). После восстановления связи сформируйте
его повторно.");
        return 2;
    }

    return 0;
}
catch (OperationCanceledException)
{
    log.Error("Формирование отчёта прервано оператором (Ctrl+C)");
    return 1;
}
catch (InvalidOperationException ex)
{
    log.Error(ex.Message);
    return 1;
}
catch (UnauthorizedAccessException ex)
{
    log.Error($"Нет прав на запись в папку отчётов: {ex.Message}");
    return 1;
}
catch (IOException ex)
{
    log.Error($"Ошибка записи файла (файл открыт в Excel?): {ex.Message}");
    return 1;
}
```

Листинг 2 – MockServerClient.cs — HttpClient, таймауты, повторы, кэш

```
using System.Diagnostics;
using System.Net;
using System.Text.Json;

namespace Campus.MealDailyReport;

/// <summary>Результат выгрузки: записи источника и признак «данные взяты из
кэша».</summary>
public sealed record LoadResult(IReadOnlyList<SourcePost> Posts, bool FromCache, DateTime
LoadedAt, long LatencyMs);

/// <summary>
/// Клиент публичного мок-сервера (выгрузка транзакций кассы).
/// Обработка сетевых ошибок по методическим указаниям: try-catch, таймауты,
/// повторные попытки с увеличением паузы; при полной недоступности — кэш последней
выгрузки,
/// чтобы бухгалтерия получила хотя бы предварительный отчёт.
```

```
/// </summary>
public sealed class MockServerClient(HttpClient http, ReportLog log, string cachePath,
int retries)
{
    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web);

    public async Task<LoadResult> LoadAsync(string url, CancellationToken ct)
    {
        for (int attempt = 1; attempt <= retries; attempt++)
        {
            var sw = Stopwatch.StartNew();
            try
            {
                log.Info($"Запрос к мок-серверу ({attempt}/{retries}): GET {url}");
                using var response = await http.GetAsync(url, ct);
                sw.Stop();

                if (response.StatusCode == HttpStatusCode.TooManyRequests ||
(int)response.StatusCode >= 500)
                {
                    // временная ошибка сервера — имеет смысл повторить
                    throw new HttpRequestException($"сервер ответил
{(int)response.StatusCode} {response.ReasonPhrase}");
                }

                if (!response.IsSuccessStatusCode)
                {
                    // 4xx — ошибка адреса/доступа, повтор не поможет
                    throw new InvalidOperationException($"Источник вернул
{(int)response.StatusCode} {response.ReasonPhrase}: проверьте SourceUrl");
                }

                string body = await response.Content.ReadAsStringAsync(ct);
                var posts = Parse(body);
                log.Ok($"Получено записей: {posts.Count}, время отклика
{sw.ElapsedMilliseconds} мс");

                // кэш последней успешной выгрузки
                Directory.CreateDirectory(Path.GetDirectoryName(cachePath)!);
                await File.WriteAllTextAsync(cachePath, body, ct);
                return new LoadResult(posts, false, DateTime.Now,
sw.ElapsedMilliseconds);
            }
            catch (TaskCanceledException) when (!ct.IsCancellationRequested)
            {
                log.Warn($"Таймаут {http.Timeout.TotalSeconds:0} с — сервер не ответил");
            }
            catch (HttpRequestException ex)
            {
                log.Warn($"Сетевая ошибка: {ex.Message}");
            }
            catch (JsonException ex)
            {
                log.Warn($"Ответ сервера не является корректным JSON: {ex.Message}");
            }

            if (attempt < retries)
            {
                var pause = TimeSpan.FromSeconds(Math.Pow(2, attempt - 1)); // 1 с, 2 с,
4 с ...
                log.Info($"Повтор через {pause.TotalSeconds:0} с");
                await Task.Delay(pause, ct);
            }
        }

        // Все попытки исчерпаны — работаем по кэшу
        if (File.Exists(cachePath))
        {
            var posts = Parse(await File.ReadAllTextAsync(cachePath, ct));
            var cachedAt = File.GetLastWriteTime(cachePath);
            log.Warn($"Мок-сервер недоступен. Используется кэш выгрузки от
{cachedAt:dd.MM.yyyy HH:mm} ({posts.Count} записей)");
            return new LoadResult(posts, true, cachedAt, -1);
        }
```

```
        throw new InvalidOperationException("Мок-сервер недоступен, кэш выгрузки
отсутствует — отчёт не может быть сформирован");
    }

    /// <summary>Разбор и валидация выгрузки: отбрасываем записи без
идентификатора.</summary>
    private static List<SourcePost> Parse(string json)
    {
        var posts = JsonSerializer.Deserialize<List<SourcePost>>(json, Json)
                    ?? throw new JsonException("пустой ответ");
        return posts.Where(p => p.Id > 0 && p.UserId > 0)
                    .Select(p => p with { Title = p.Title ?? "", Body = p.Body ?? "" })
                    .ToList();
    }
}
```

Листинг 3 – TransactionSimulator.cs — симуляция и суммирование по категориям

```
namespace Campus.MealDailyReport;

/// <summary>
/// Симуляция транзакций кассы столовой по записям мок-сервера.
/// Преобразование детерминированное: одна и та же выгрузка за одни и те же сутки
/// всегда даёт одинаковый отчёт — это важно для сверки и контрольных сумм.
/// </summary>
public static class TransactionSimulator
{
    // Прейскурант комплексов (руб.)
    private static readonly (string Name, decimal Price)[] Menu =
    [
        ("Завтрак", 95m),
        ("Обед", 185m),
        ("Полдник", 70m)
    ];

    /// <summary>Лимит дотации на одну покупку для бюджетной категории (руб.).</summary>
    public const decimal BudgetSubsidyLimit = 150m;

    private static readonly TimeSpan Opening = new(8, 0, 0);   // столовая открыта 08:00–
19:00
    private const int WorkingMinutes = 11 * 60;

    public static MealTransaction Create(SourcePost post, DateOnly date, string
stationId, string operatorId)
    {
        // Категория: пользователи 1–4 мок-сервера — студенты-бюджетники, остальные —
коммерция
        var category = post.UserId <= 4 ? MealCategory.Budget : MealCategory.Commercial;

        // Время покупки распределяется по рабочему дню столовой
        var time = Opening + TimeSpan.FromMinutes(post.Id * 37 % WorkingMinutes);
        var timestamp = date.ToDateTime(TimeOnly.FromTimeSpan(time));

        // Тип комплекса зависит от времени: до 11:00 — завтрак, 11:00–15:00 — обед,
позже — полдник
        var meal = time.Hours < 11 ? Menu[0] : time.Hours < 15 ? Menu[1] : Menu[2];

        // Доп. позиции (напиток, выпечка): 0–4 позиции по 15 руб. — от длины текста
записи
        decimal amount = meal.Price + post.Body.Length % 5 * 15m;

        // Статус: каждая 17-я запись — отказ, каждая 23-я — возврат
        var status = post.Id % 17 == 0 ? TransactionStatus.Declined
                   : post.Id % 23 == 0 ? TransactionStatus.Refund
                   : TransactionStatus.Success;

        // Дотация только для бюджета и только по проведённым (не отклонённым) операциям
        decimal subsidy = category == MealCategory.Budget && status !=
TransactionStatus.Declined
            ? Math.Min(amount, BudgetSubsidyLimit)
            : 0m;

        return new MealTransaction(
            TransactionId: $"TX-{date:yyyyMMdd}-{post.Id:0000}",
```

```
            Timestamp: timestamp,
            StationId: stationId,
            OperatorId: operatorId,
            Category: category,
            MealType: meal.Name,
            Amount: amount,
            Subsidy: subsidy,
            Status: status,
            SourceRecordId: post.Id);
    }

    /// <summary>Суммирование финансовых показателей по категориям.</summary>
    public static IReadOnlyList<CategoryTotals> Summarize(IEnumerable<MealTransaction>
transactions) =>
        transactions
            .GroupBy(t => t.Category)
            .OrderBy(g => g.Key)
            .Select(g => new CategoryTotals(
                Category: g.Key,
                Success: g.Count(t => t.Status == TransactionStatus.Success),
                Refunds: g.Count(t => t.Status == TransactionStatus.Refund),
                Declined: g.Count(t => t.Status == TransactionStatus.Declined),
                // продажи: проведённые оплаты + оплаты, по которым потом был возврат
                Revenue: g.Where(t => t.Status != TransactionStatus.Declined).Sum(t =>
t.Amount),
                RefundAmount: g.Where(t => t.Status == TransactionStatus.Refund).Sum(t =>
t.Amount),
                Subsidy: g.Where(t => t.Status == TransactionStatus.Success).Sum(t =>
t.Subsidy),
                Paid: g.Where(t => t.Status == TransactionStatus.Success).Sum(t =>
t.ToPay)))
            .ToList();
}
```

Листинг 4 – ReportWriter.cs — CSV, Markdown и SHA-256

```
using System.Globalization;
using System.Security.Cryptography;
using System.Text;

namespace Campus.MealDailyReport;

/// <summary>
/// Формирование файлов суточного отчёта (подход Docs-as-Code: результат — текстовые
файлы,
/// которые можно хранить в Git, сравнивать и открывать в Excel):
///   * meal_transactions_ГГГГММДД.csv  — реестр транзакций за сутки;
///   * meal_daily_report_ГГГГММДД.csv  — финансовые итоги по категориям;
///   * meal_daily_report_ГГГГММДД.md   — сводка для руководства/бухгалтерии;
///   * meal_daily_report_ГГГГММДД.sha256 — контрольные суммы (защита от изменений).
/// CSV — с разделителем «;», UTF-8 с BOM и запятой в дробных числах: так файл
/// корректно открывается двойным щелчком в русской версии Excel.
/// </summary>
public sealed class ReportWriter(string directory)
{
    private static readonly CultureInfo Ru = CultureInfo.GetCultureInfo("ru-RU");
    private static readonly Encoding Utf8Bom = new UTF8Encoding(true);
    private readonly List<string> _written = [];

    public string Directory { get; } = directory;

    public string WriteTransactions(string fileName, IEnumerable<MealTransaction>
transactions)
    {
        var sb = new StringBuilder();

sb.AppendLine("Date;Time;TransactionId;StationId;OperatorId;Category;MealType;Amount;Subs
idy;ToPay;Transaction_Status");
        foreach (var t in transactions)
        {
            sb.AppendLine(string.Join(';',
                t.Timestamp.ToString("dd.MM.yyyy", Ru),
                t.Timestamp.ToString("HH:mm", Ru),
                t.TransactionId,
```

```
                Csv(t.StationId),
                Csv(t.OperatorId),
                t.Category.ToRussian(),
                t.MealType,
                Money(t.Amount),
                Money(t.Subsidy),
                Money(t.ToPay),
                t.Status.ToRussian()));
        }

        return Save(fileName, sb.ToString());
    }

    public string WriteCategoryReport(string fileName, IReadOnlyList<CategoryTotals>
totals, string header)
    {
        var sb = new StringBuilder();
        sb.AppendLine(Csv(header));

sb.AppendLine("Category;Success;Refunds;Declined;Revenue;RefundAmount;NetRevenue;Subsidy;
Paid;AverageCheck");
        foreach (var c in totals)
        {
            sb.AppendLine(string.Join(';', c.Category.ToRussian(), c.Success, c.Refunds,
c.Declined,
                Money(c.Revenue), Money(c.RefundAmount), Money(c.NetRevenue),
Money(c.Subsidy), Money(c.Paid), Money(c.AverageCheck)));
        }

        // итоговая строка — для сверки с Z-отчётом кассы
        sb.AppendLine(string.Join(';', "ИТОГО",
            totals.Sum(c => c.Success), totals.Sum(c => c.Refunds), totals.Sum(c =>
c.Declined),
            Money(totals.Sum(c => c.Revenue)), Money(totals.Sum(c => c.RefundAmount)),
Money(totals.Sum(c => c.NetRevenue)),
            Money(totals.Sum(c => c.Subsidy)), Money(totals.Sum(c => c.Paid)), ""));

        return Save(fileName, sb.ToString());
    }

    public string WriteMarkdownSummary(string fileName, AppConfig cfg, LoadResult loaded,
IReadOnlyList<CategoryTotals> totals, int count)
    {
        var sb = new StringBuilder();
        sb.AppendLine($"# Суточный отчёт по транзакциям питания —
{cfg.ReportDate:dd.MM.yyyy}");
        sb.AppendLine();
        sb.AppendLine("| Параметр | Значение |");
        sb.AppendLine("|---|---|");
        sb.AppendLine($"| ИС | «Умный кампус», модуль «Столовая» |");
        sb.AppendLine($"| АРМ (StationId) | `{cfg.StationId}` |");
        sb.AppendLine($"| Оператор | `{cfg.OperatorId}` |");
        sb.AppendLine($"| Сформирован | {DateTime.Now:dd.MM.yyyy HH:mm:ss} |");
        sb.AppendLine($"| Источник | {(loaded.FromCache ? $"**КЭШ** выгрузки от
{loaded.LoadedAt:dd.MM.yyyy HH:mm}" : cfg.SourceUrl)} |");
        sb.AppendLine($"| Транзакций | {count} |");
        sb.AppendLine();
        sb.AppendLine("## Итоги по категориям");
        sb.AppendLine();
        sb.AppendLine("| Категория | Успешно | Возвраты | Отказы | Чистая выручка, ₽ |
Дотация, ₽ | Оплачено, ₽ | Средний чек, ₽ |");
        sb.AppendLine("|---|---:|---:|---:|---:|---:|---:|---:|");
        foreach (var c in totals)
        {
            sb.AppendLine($"| {c.Category.ToRussian()} | {c.Success} | {c.Refunds} |
{c.Declined} | {Money(c.NetRevenue)} | {Money(c.Subsidy)} | {Money(c.Paid)} |
{Money(c.AverageCheck)} |");
        }

        sb.AppendLine($"| **ИТОГО** | **{totals.Sum(c => c.Success)}** | **{totals.Sum(c
=> c.Refunds)}** | **{totals.Sum(c => c.Declined)}** | " +
                      $"**{Money(totals.Sum(c => c.NetRevenue))}** |
**{Money(totals.Sum(c => c.Subsidy))}** | **{Money(totals.Sum(c => c.Paid))}** | |");
        sb.AppendLine();
        sb.AppendLine("*Отчёт сформирован автоматически утилитой campus-meal-daily-report
```

```
1.0.0. " +
                      "Целостность файлов проверяется по контрольным суммам SHA-256.*");
        return Save(fileName, sb.ToString());
    }

    /// <summary>Контрольные суммы всех файлов отчёта (формат утилиты
sha256sum).</summary>
    public string WriteChecksums(string fileName)
    {
        var sb = new StringBuilder();
        foreach (string path in _written)
        {
            using var stream = File.OpenRead(path);
            string hash =
Convert.ToHexString(SHA256.HashData(stream)).ToLowerInvariant();
            sb.AppendLine($"{hash}  {Path.GetFileName(path)}");
        }

        string target = Path.Combine(Directory, fileName);
        File.WriteAllText(target, sb.ToString(), new UTF8Encoding(false));
        return target;
    }

    private string Save(string fileName, string content)
    {
        System.IO.Directory.CreateDirectory(Directory);
        string path = Path.Combine(Directory, fileName);
        File.WriteAllText(path, content, Utf8Bom);
        _written.Add(path);
        return path;
    }

    private static string Money(decimal value) => value.ToString("0.00", Ru);

    /// <summary>Экранирование значения CSV (кавычки, «;» и переводы строк).</summary>
    private static string Csv(string value) =>
        value.IndexOfAny([';', '"', '\n', '\r']) >= 0 ? $"\"{value.Replace("\"",
"\"\"")}\"" : value;
}
```

Листинг 5 – AppConfig.cs — иерархия конфигурации

```
using System.Globalization;
using System.Text.Json;

namespace Campus.MealDailyReport;

/// <summary>
/// Параметры запуска утилиты. Собираются по иерархии конфигурации .NET:
///   1) appsettings.json        — базовые статические настройки (URL, таймауты);
///   2) переменные окружения    — CAMPUS_STATION_ID, CAMPUS_OPERATOR_ID,
CAMPUS_SOURCE_URL
///                                (настройки конкретного АРМ, задаются администратором);
///   3) аргументы командной строки — наивысший приоритет (--date, --source, --out, --
station, --operator).
/// Каждый следующий уровень переопределяет предыдущий.
/// </summary>
public sealed class AppConfig
{
    public string SourceUrl { get; set; } = "https://jsonplaceholder.typicode.com/posts";
    public string OutputDir { get; set; } = "reports";
    public int TimeoutSeconds { get; set; } = 10;
    public int Retries { get; set; } = 3;
    public string StationId { get; set; } = "CANTEEN-A1-POS-01";

    /// <summary>Оператор, сформировавший отчёт (полнота данных: кто и где
сформировал).</summary>
    public string OperatorId { get; set; } = Environment.UserName;

    /// <summary>Отчётные сутки.</summary>
    public DateOnly ReportDate { get; set; } = DateOnly.FromDateTime(DateTime.Today);

    public static AppConfig Build(string baseDirectory, string[] args)
    {
```

```
        // ---- 1. appsettings.json ------------------------------------------
        string path = Path.Combine(baseDirectory, "appsettings.json");
        var config = File.Exists(path)
            ? JsonSerializer.Deserialize<AppConfig>(File.ReadAllText(path), new
JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true,
                ReadCommentHandling = JsonCommentHandling.Skip,
                AllowTrailingCommas = true
            }) ?? new AppConfig()
            : new AppConfig();

        // ---- 2. переменные окружения с префиксом CAMPUS_ ---------------------
        config.StationId = Env("CAMPUS_STATION_ID") ?? config.StationId;
        config.OperatorId = Env("CAMPUS_OPERATOR_ID") ?? config.OperatorId;
        config.SourceUrl = Env("CAMPUS_SOURCE_URL") ?? config.SourceUrl;

        // ---- 3. аргументы командной строки -----------------------------------
        for (int i = 0; i + 1 < args.Length; i += 2)
        {
            string key = args[i].TrimStart('-').ToLowerInvariant();
            string value = args[i + 1];
            switch (key)
            {
                case "date":
                    if (!DateOnly.TryParseExact(value, "yyyy-MM-dd",
CultureInfo.InvariantCulture, DateTimeStyles.None, out var date))
                    {
                        throw new ArgumentException($"Параметр --date задаётся в формате
ГГГГ-ММ-ДД, получено: «{value}»");
                    }
                    config.ReportDate = date;
                    break;
                case "source": config.SourceUrl = value; break;
                case "out": config.OutputDir = value; break;
                case "station": config.StationId = value; break;
                case "operator": config.OperatorId = value; break;
                default: throw new ArgumentException($"Неизвестный параметр: {args[i]}");
            }
        }

        if (config.TimeoutSeconds <= 0 || config.Retries <= 0)
        {
            throw new ArgumentException("TimeoutSeconds и Retries в appsettings.json
должны быть больше нуля");
        }

        return config;
    }

    private static string? Env(string name)
    {
        string? value = Environment.GetEnvironmentVariable(name);
        return string.IsNullOrWhiteSpace(value) ? null : value.Trim();
    }
}
```

Листинг 6 – MealTransaction.cs — модели данных

```
namespace Campus.MealDailyReport;

/// <summary>Запись публичного мок-сервера jsonplaceholder (/posts) — «сырая» строка
выгрузки кассы.</summary>
public sealed record SourcePost(int UserId, int Id, string Title, string Body);

/// <summary>Категория питания (по отчётной форме «Ведомость учёта питания»: бюджет /
коммерция).</summary>
public enum MealCategory
{
    Budget,     // бюджетное (льготное) питание, часть стоимости покрывает дотация
    Commercial  // коммерческое питание, оплата полностью за счёт покупателя
}

/// <summary>Статус транзакции на кассе (поле Transaction_Status отчётной
```

```
формы).</summary>
public enum TransactionStatus
{
    Success,   // оплата проведена
    Refund,    // возврат (отмена покупки кассиром)
    Declined   // отказ: недостаточно средств / лимит льготы исчерпан
}

/// <summary>
/// Транзакция питания — одна строка ведомости. Поля соответствуют требованиям к
отчётности CALS:
/// точная метка времени, идентификатор АРМ и оператора, категория, сумма и статус.
/// </summary>
public sealed record MealTransaction(
    string TransactionId,
    DateTime Timestamp,
    string StationId,
    string OperatorId,
    MealCategory Category,
    string MealType,
    decimal Amount,
    decimal Subsidy,
    TransactionStatus Status,
    int SourceRecordId)
{
    /// <summary>Сумма к оплате покупателем (стоимость минус дотация).</summary>
    public decimal ToPay => Amount - Subsidy;
}

/// <summary>Финансовые итоги по одной категории за сутки.</summary>
public sealed record CategoryTotals(
    MealCategory Category,
    int Success,
    int Refunds,
    int Declined,
    decimal Revenue,
    decimal RefundAmount,
    decimal Subsidy,
    decimal Paid)
{
    /// <summary>Чистая выручка = продажи минус возвраты.</summary>
    public decimal NetRevenue => Revenue - RefundAmount;

    /// <summary>Средний чек по успешным оплатам (без учёта возвращённых).</summary>
    public decimal AverageCheck => Success == 0 ? 0 : Math.Round(NetRevenue / Success,
2);
}

public static class MealCategoryNames
{
    public static string ToRussian(this MealCategory c) => c switch
    {
        MealCategory.Budget => "Бюджет",
        MealCategory.Commercial => "Коммерция",
        _ => c.ToString()
    };

    public static string ToRussian(this TransactionStatus s) => s switch
    {
        TransactionStatus.Success => "Успешно",
        TransactionStatus.Refund => "Возврат",
        TransactionStatus.Declined => "Отклонено",
        _ => s.ToString()
    };
}
```

Листинг 7 – ReportLog.cs — журнал работы

```
using System.Text;

namespace Campus.MealDailyReport;

/// <summary>
/// Журнал работы утилиты: цветной вывод в консоль оператора + запись в файл.
```

```
/// Файл открывается только на дозапись (append-only): строки прошлых запусков не
изменяются,
/// что соответствует требованию «защита от изменений» для отчётности кампуса.
/// </summary>
public sealed class ReportLog : IDisposable
{
    private readonly StreamWriter _file;

    public ReportLog(string directory)
    {
        Directory.CreateDirectory(directory);
        FilePath = Path.Combine(directory, "meal_report.log");
        _file = new StreamWriter(new FileStream(FilePath, FileMode.Append,
FileAccess.Write, FileShare.Read), new UTF8Encoding(false))
        {
            AutoFlush = true
        };
    }

    public string FilePath { get; }

    public void Info(string message) => Write("INF", message, ConsoleColor.Gray);
    public void Ok(string message) => Write("OK ", message, ConsoleColor.Green);
    public void Warn(string message) => Write("WRN", message, ConsoleColor.Yellow);
    public void Error(string message) => Write("ERR", message, ConsoleColor.Red);

    /// <summary>Строка без метки уровня (таблицы итогов, заголовки).</summary>
    public void Raw(string text, ConsoleColor color = ConsoleColor.Gray)
    {
        Console.ForegroundColor = color;
        Console.WriteLine(text);
        Console.ResetColor();
        _file.WriteLine(text);
    }

    private void Write(string level, string message, ConsoleColor color)
    {
        string line = $"{DateTime.Now:yyyy-MM-dd HH:mm:ss.fff} [{level}] {message}";
        Console.ForegroundColor = color;
        Console.WriteLine(line);
        Console.ResetColor();
        _file.WriteLine(line);
    }

    public void Dispose() => _file.Dispose();
}
```

Листинг 8 – appsettings.json

```
{
  // Публичный мок-сервер по варианту 10 — имитирует выгрузку транзакций с кассы столовой
  "SourceUrl": "https://jsonplaceholder.typicode.com/posts",
  // Папка отчётов (относительно папки запуска). Внутри создаётся подпапка на каждые
сутки
  "OutputDir": "reports",
  // Сетевые параметры: таймаут одного запроса и число повторных попыток
  "TimeoutSeconds": 10,
  "Retries": 3,
  // Идентификатор АРМ кассы. Переопределяется переменной окружения CAMPUS_STATION_ID
  "StationId": "CANTEEN-A1-POS-01"
}
```

4. Снимки экрана

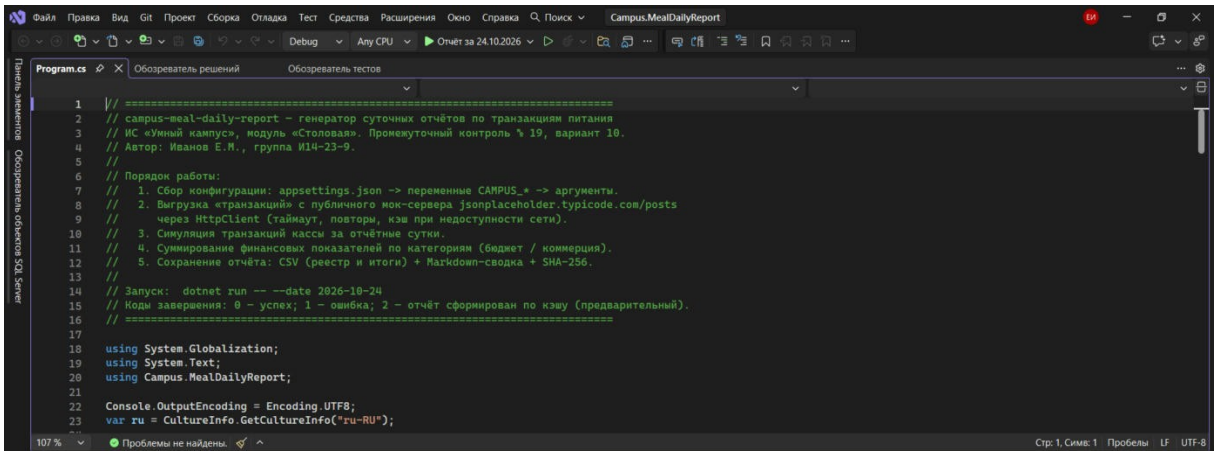


Рисунок 1 – Проект Campus.MealDailyReport в Visual Studio

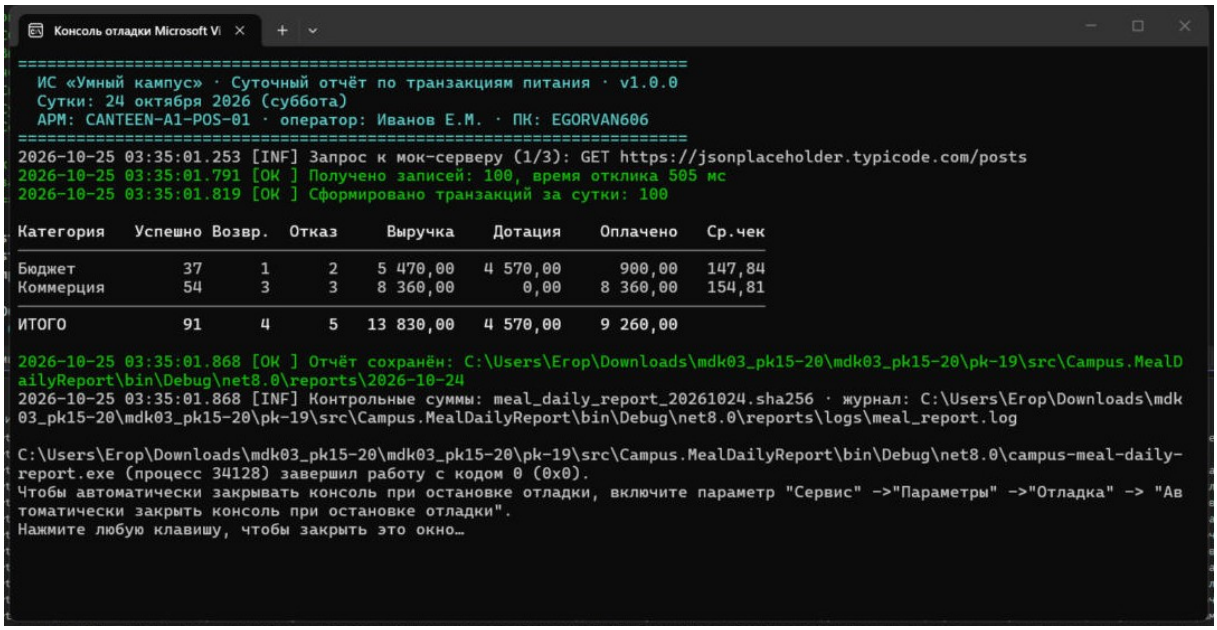


Рисунок 2 – Запуск утилиты: выгрузка с jsonplaceholder.typicode.com, итоги по категориям «Бюджет» и «Коммерция»

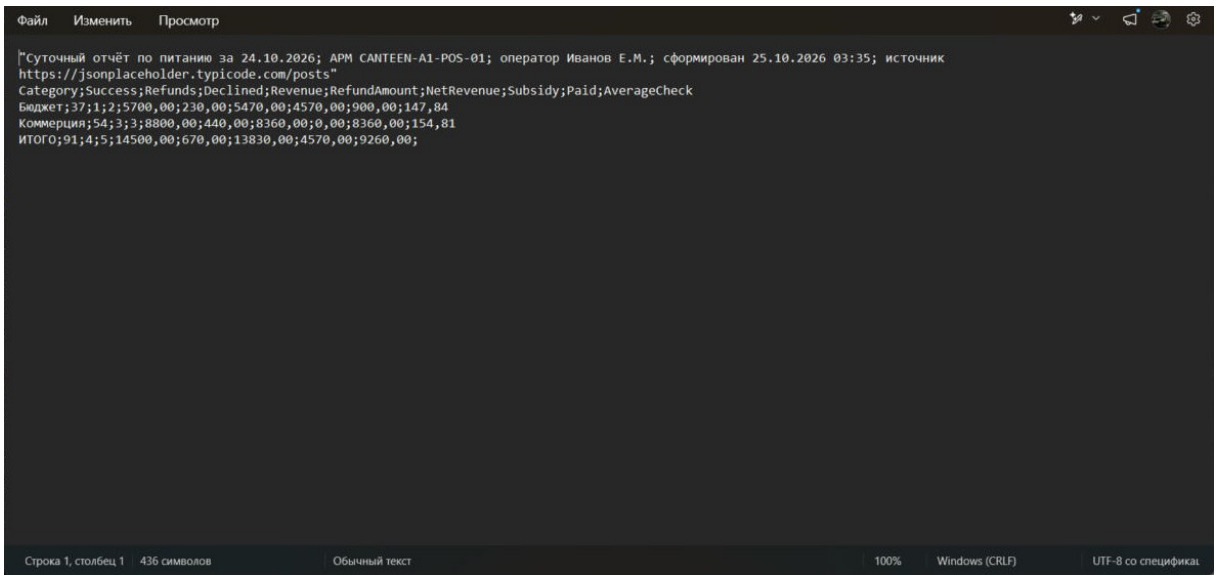


Рисунок 3 – Сформированный суточный отчёт meal_daily_report_20261024.csv

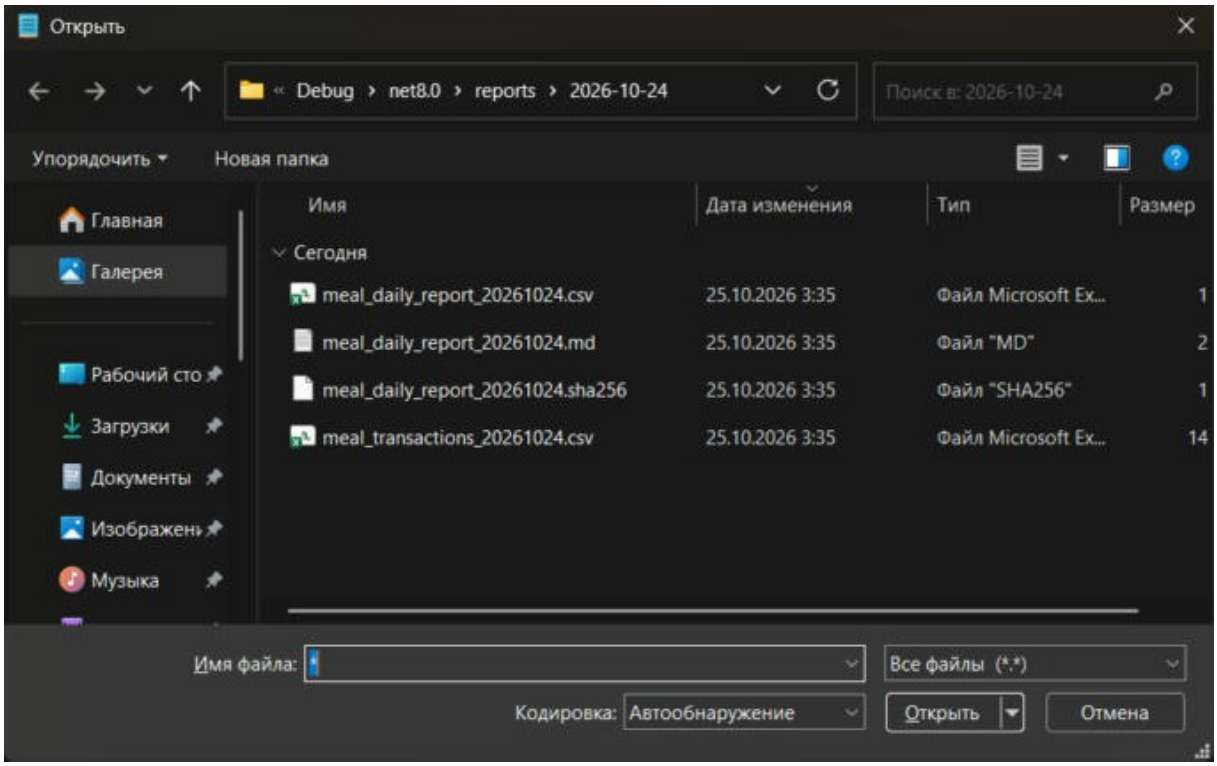


Рисунок 4 – Папка отчёта за сутки: реестр CSV, итоги CSV, сводка Markdown, контрольные суммы SHA-256

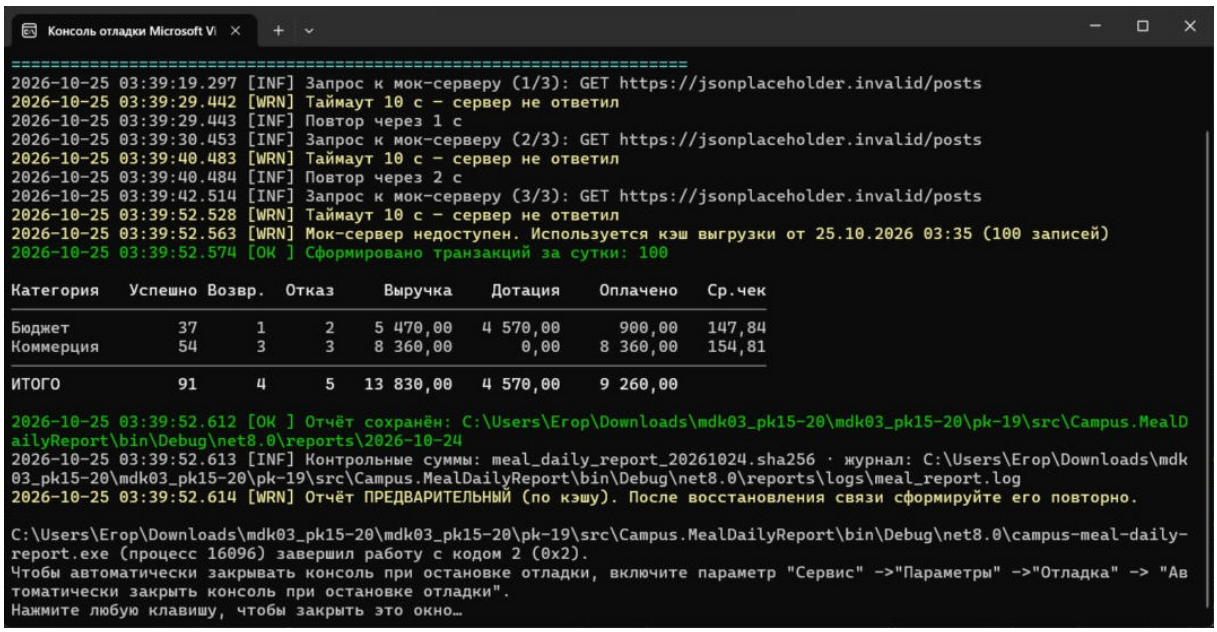


Рисунок 5 – Сбой сети (неверный адрес API): три попытки с паузой и формирование предварительного отчёта по кэшу

5. Вывод

Разработана консольная утилита на C# .NET 8 для АРМ кассы столовой ИС «Умный кампус». Она загружает выгрузку с публичного мок-сервера через `HttpClient`, симулирует транзакции питания за сутки, суммирует финансовые показатели по категориям «Бюджет» и «Коммерция» (выручка, возвраты, дотация, оплачено, средний чек) и сохраняет отчёт в CSV и Markdown.

По методическим указаниям обрабатываются сетевые ошибки: таймаут, повторные попытки и предварительный отчёт по кэшу при недоступности сервера. По требованиям лекции к отчётности кампуса в каждой строке есть метки времени, АРМ и оператора, целостность файлов подтверждают контрольные суммы SHA-256, а конфигурация собирается по иерархии appsettings.json → переменные окружения → аргументы. Результат — текстовые файлы, которые хранятся в Git и открываются в Excel, что соответствует подходу Docs-as-Code.